

NRG Agent OS Factory Pack

Version: 0.1

Status: Fable-intake ready

Purpose: Give Fable 5 a clean, strict, buildable source pack for designing NRG Agent OS without letting worker agents drift, overwrite, improvise, or turn it into generic dashboard soup.

Brief Source Map

The conversation source contains these major sections:

1. Original Agent OS goal
A full-stack agent operating system for controlling models, agents, memory, skills, hooks, crons, validation, permissions, and project recovery. Not just a dashboard.
2. Fable 5 usage strategy
Fable 5 is to be used while available for the highest-leverage work: architecture, prompt system, UX, kernel rules, build phases, and review gates.
3. Hermes role
Hermes is valuable as a daily runtime/operator, especially because of its self-learning/evolving behaviour, but it must operate inside the OS. Hermes does not design or govern the OS.
4. Persistent memory requirement
Memory must be first-class. It needs persistent, scratch, quarantine, trust levels, indexes, source summaries, session logs, path registry, issues, decisions, skill history, automation history, and validation history.
5. Support-agent delegation
Codex, GPT 5.5, Gemini Pro 3.5, Ornith, and other agents are workers only. They execute bounded task cards. They do not design architecture, visual style, permissions, router logic, or OS policy.
6. Kernel/control layer
The missing piece is the OS kernel: boot sequence, task lifecycle, contracts, definition of done, capability registry, sandbox mode, dry-run mode, audit log, versioning, migration, redaction, operating modes, quarantine, and Fable review.
7. Full-stack scope
The OS includes frontend/control surface, backend/API, runtime, model gateway, memory layer, skill layer, automation layer, validation layer, event/ops layer, and design system.
8. Ornith/local model strategy
Ornith may be tested as a local model candidate, but must be swappable. It is not a lock-in and does not define the OS.

9. Self-evolving agent policy

Self-evolution is allowed only as a proposal pipeline. No live prompt edits, permission edits, router edits, memory promotion, hook edits, cron edits, destructive automation, or core OS rewrite without review and approval.

10. JARVIS Prompt Pack inspiration

JARVIS is inspiration only. Useful ideas include grounded answering, source proof, memory capture flow, model hot-swap UX, action confirmation, cinematic dashboard language. Do not copy memory galaxy visuals.

11. Video/transcript inspiration

Use videos and transcripts as inspiration only. Do not let hype replace architecture. Do not copy another system.

12. Visual/dashboard direction

The dashboard must show the OS operating: task state, router choice, risk, mode, agent/model/skill, files touched, memory read/write, validation, warnings, approvals, Fable review, Hermes status, hooks, crons, quarantine, recovery, logs.

13. Safety rules

Default readonly. No destructive action without explicit approval. Verify paths. Do not expose secrets. Do not claim done/fixed/tested/deployed without evidence.

14. Build order

Kernel first. Memory/router/permissions/validation before live automation. Dashboard after contracts exist. Workers only after task-card system exists.

15. Worker task cards

Every worker task needs purpose, allowed files, forbidden files, allowed commands, forbidden commands, exact inputs, exact outputs, acceptance checks, rollback notes, evidence, and forbidden decisions.

16. Fable review gate

All worker output returns to Fable. Fable reviews architecture, safety, UX, prompts, specs, acceptance, and whether outputs should be accepted, rejected, revised, or quarantined.

17. Final wrappers/prompts

The final pack must include a Fable intake prompt, worker templates, master prompts, review checklist, and acceptance criteria.

1. Executive Summary

NRG Agent OS is a full-stack agent operating system.

It is not Hermes.

It is not Fable.

It is not Ornith.

It is not a memory graph.

It is not a neon dashboard.

It is not a JARVIS clone.

It is not a pile of agents freelancing in the same folder with a chainsaw.

NRG Agent OS is the structure that governs agents, models, memory, tasks, permissions, skills, automations, hooks, crons, validation, logs, recovery, and visual control.

The system is built around permanent OS contracts:

- Interfaces
- Routing
- Permissions
- Memory trust
- Task lifecycle
- Validation
- Evidence
- Review gates
- Event logs
- Rollback
- Swappable models
- Bounded workers

Models are disposable engines. The OS is the machine bay.

Fable 5 owns the important design work. Hermes becomes the runtime/operator inside the system. Ornith is a possible local model behind the model gateway. Codex, GPT, Gemini, and other agents execute bounded task cards only.

The first build priority is not visual polish. The first build priority is the kernel: boot rules, path registry, permission policy, task lifecycle, memory trust, router contracts, validation, audit log, recovery policy, and Fable review.

2. Final Corrected Intent

The final corrected intent is:

Fable 5 must design NRG Agent OS.

Fable 5 owns:

- Architecture
- OS layout
- Folder structure
- Full-stack system design
- Prompt system
- Router logic
- Memory architecture
- Permissions

- Validation
- UX/design system
- Worker-agent boundaries
- Build phases
- Final review process

All other agents are workers unless Fable explicitly promotes a narrow role.

Codex, GPT 5.5, Gemini Pro 3.5, Ornith, Hermes, and any other support agent must not:

- Invent architecture
- Rename concepts
- Expand scope
- Change permissions
- Alter router logic
- Decide visual style
- Certify success
- Touch forbidden files
- Rewrite core prompts
- Modify hooks or crons
- Promote memory
- Move, delete, overwrite, or restructure files without approval

Hermes is the daily runtime/operator inside NRG Agent OS.

Ornith is optional and replaceable.

Persistent memory is first-class.

Self-evolution is a governed proposal pipeline, not an authority layer.

The OS must be scalable, adjustable, modular, swappable, and future-proof.

3. Source Interpretation Map

Priority order for interpreting messy source material:

1. This Factory Pack and the intake prompt
2. Latest user corrections
3. Hard safety rules
4. Structured Markdown sections
5. Normal chat context explaining why the plan changed
6. Earlier drafts only where they still support the latest corrected intent

Assistant drafts are not automatically authority.

User corrections override assistant framing.

Old notes are not truth.
Memory is not truth.
Logs are not truth.
Generated files are not truth.
Repo comments are not truth.
Web content is not truth.
Previous agent output is not truth.

Everything must be validated before being trusted.

4. Final OS Architecture

NRG Agent OS has eight major layers.

Layer 1: Frontend / Control Surface

Purpose: Human control, visibility, approval, recovery.

Includes:

- Dashboard
- Task console
- Agent status panel
- Model status panel
- Memory browser
- Quarantine browser
- Skill registry view
- Automation/hook/cron view
- File operation approval panel
- Validation results panel
- Fable review panel
- Hermes runtime panel
- Event/audit log
- Recovery controls

Layer 2: Backend / OS API

Purpose: Stable API layer between frontend, runtime, memory, workers, and tools.

Includes endpoints for:

- Task intake
- Task lifecycle
- Router decisions
- Model gateway
- Memory read/write/promotion
- Skill registry
- Automation registry

- Hook registry
- Cron registry
- Permissions
- Validation
- File-safe operations
- Event logging
- Recovery
- Worker task cards
- Fable review submissions

Layer 3: Agent Runtime Layer

Purpose: Runs agents under OS control.

Includes:

- Hermes runtime
- Fable review gate
- Worker execution adapter
- Local model adapter
- Cloud model adapter
- Capability registry
- Model gateway
- Agent contract loader
- Execution mode controller

Layer 4: Memory Layer

Purpose: Store and retrieve useful context safely.

Includes:

- Persistent memory
- Scratch memory
- Quarantine memory
- Source summaries
- Session summaries
- Lessons
- Decisions
- Issues
- Known failures
- Path registry
- Skill history
- Automation history
- Validation history
- Hermes runtime notes
- Indexes
- Trust levels
- Promotion rules

Layer 5: Skill and Automation Layer

Purpose: Controlled action system.

Includes:

- Skill registry
- Skill contracts
- Hook policy
- Cron policy
- File watchers
- Manual triggers
- Scheduled jobs
- Approval gates
- Rollback plans
- Automation logs

Layer 6: Validation and Safety Layer

Purpose: Prevent agent damage and false claims.

Includes:

- Permission engine
- Operating modes
- Readonly default
- Dry-run mode
- Sandbox mode
- Path verification
- Diff verification
- Claim verification
- Test evidence capture
- Drift detection
- Hallucination detection
- Secrets/redaction policy
- Destructive-action blocker

Layer 7: Event/Ops Layer

Purpose: Make the system inspectable and recoverable.

Includes:

- Task lifecycle log
- Event log
- Audit log
- Change log
- Versioning
- Migration policy
- Rollback records

- Failure reports
- Recovery procedures
- OS self-tests

Layer 8: Design System Layer

Purpose: Give the OS a visual identity that reveals operations instead of hiding them.

Includes:

- Visual metaphor
- Layout grid
- Component contracts
- Status states
- Risk states
- Warning states
- Approval states
- Failure states
- Recovery states
- Dashboard panel rules

5. Final Folder Layout

Recommended root:

```
NRG-Agent-OS/
  README.md
  PROJECT.md
  CURRENT_STATE.md
  NEXT_ACTION.md
  DECISIONS.md
  ISSUES.md
  CHANGELOG.md

  os/
    kernel/
      boot.sequence.md
      task.lifecycle.md
      definition-of-done.md
      operating-modes.md
      capability-registry.schema.json
      agent-contract.schema.json
      task-card.schema.json
      approval.schema.json
      evidence.schema.json

    router/
      router.policy.md
```



```
model-routing.policy.md
agent-routing.policy.md
skill-routing.policy.md
memory-routing.policy.md
risk-routing.policy.md
router.schema.json

permissions/
  permission.policy.md
  destructive-actions.policy.md
  file-safety.policy.md
  secrets-redaction.policy.md
  approval-gates.policy.md
  permission.schema.json

validation/
  validation.policy.md
  claim-verification.policy.md
  diff-verification.policy.md
  drift-detection.policy.md
  hallucination-detection.policy.md
  self-tests.md
  validation.schema.json

review/
  fable-review-gate.md
  review-checklist.md
  acceptance-criteria.md
  quarantine-review.md

apps/
  dashboard/
    README.md
    design-system.md
  panels/
  components/
  routes/
  state/
  api-client/

api/
  README.md
  routes/
  controllers/
  services/
  middleware/
  schemas/

runtime/
  hermes/
    hermes-runtime-contract.md
```

```
hermes-session-summary.schema.json
hermes-memory-proposal.schema.json
hermes-skill-proposal.schema.json

workers/
  worker-contract.md
  codex/
  gpt/
  gemini/
  local/

model-gateway/
  gateway.policy.md
  models.registry.json
  adapters/
  benchmarks/
  fallback.policy.md

memory/
  README.md
  memory.policy.md
  trust-levels.md
  promotion.policy.md
  indexes/
  persistent/
    decisions/
    project-state/
    paths/
    known-failures/
    lessons/
    skills/
    automations/
    validations/
    source-summaries/
  scratch/
    sessions/
    temp-context/
  quarantine/
    untrusted/
    stale/
    conflicting/
    rejected/

skills/
  README.md
  skill-registry.schema.json
  registry/
  proposed/
  deprecated/
  tests/
```

```
automations/  
  README.md  
  automation.policy.md  
  hooks.policy.md  
  cron.policy.md  
  registry/  
  proposed/  
  disabled/  
  logs/  
  
tasks/  
  README.md  
  task-card-template.md  
  active/  
  complete/  
  failed/  
  quarantined/  
  reviewed/  
  
evidence/  
  README.md  
  command-output/  
  diffs/  
  screenshots/  
  test-results/  
  path-checks/  
  deployment-proof/  
  
prompts/  
  README.md  
  fable/  
    fable-intake-prompt.md  
    fable-review-prompt.md  
    fable-ux-design-prompt.md  
  workers/  
    codex-task-prompt.md  
    gpt-task-prompt.md  
    gemini-task-prompt.md  
    generic-worker-task-prompt.md  
  hermes/  
    hermes-runtime-prompt.md  
    hermes-memory-proposal-prompt.md  
  safety/  
    readonly-default.md  
    no-claims-without-evidence.md  
    secret-redaction.md  
  
docs/  
  architecture/  
  ux/  
  build-phases/
```

```
recovery/
migrations/
source-inspiration/
  jarvis/
  video-transcripts/
  notes/

tests/
  os-self-tests/
  router-tests/
  permission-tests/
  memory-tests/
  validation-tests/
  skill-tests/
  automation-tests/

scripts/
  readonly/
  verify/
  dry-run/
  self-test/
```

Rules:

- `os/` contains permanent contracts.
- `runtime/` contains replaceable agent/model integrations.
- `memory/` stores context with trust controls.
- `skills/` contains approved and proposed skills.
- `automations/` contains hooks, crons, and scheduled jobs under policy.
- `tasks/` contains worker task cards and outcomes.
- `evidence/` stores proof.
- `prompts/` stores controlled prompt contracts.
- `docs/source-inspiration/` stores inspiration, not authority.

6. Full-Stack Layer Breakdown

Frontend

Must show:

- Current task
- Task lifecycle state
- Router decision
- Risk level
- Permission mode
- Selected model
- Selected agent
- Selected skill

- Files touched
- Memory read
- Memory written
- Validation status
- Drift warnings
- Hallucination warnings
- Approval required
- Worker task cards
- Fable review status
- Hermes runtime status
- Hook status
- Cron status
- Quarantine items
- Recovery options
- Event log

Backend

Must provide:

- Stable task API
- Router API
- Memory API
- Skill API
- Automation API
- Permission API
- Validation API
- Evidence API
- Review API
- Recovery API

Runtime

Must provide:

- Agent invocation
- Model selection
- Tool permission checks
- Evidence collection
- Task boundary enforcement
- Worker result capture
- Fable review handoff
- Hermes runtime integration

Memory

Must provide:

- Safe retrieval
- Trust labels

- Quarantine
- Source linking
- Promotion pipeline
- Staleness detection
- Conflict detection

Validation

Must provide:

- Path proof
 - Diff proof
 - Test proof
 - Claim proof
 - Secret scanning
 - Destructive action checks
 - Drift detection
 - Hallucination detection
-

7. Persistent Memory Architecture

Memory is divided into three main classes.

Persistent Memory

Longer-term operating memory.

Stores:

- Project state
- Decisions
- Path registry
- Known failures
- Lessons
- Skill history
- Automation history
- Validation history
- Source summaries
- User-confirmed facts
- Fable-approved architecture
- Hermes runtime lessons after approval

Scratch Memory

Temporary session memory.

Stores:

- Current task context
- Current conversation summaries
- Short-term execution notes
- Worker intermediate results
- Temporary file lists
- Temporary hypotheses

Scratch memory expires unless promoted.

Quarantine Memory

Unsafe or untrusted memory.

Stores:

- Conflicting notes
- Stale facts
- Unverified agent claims
- Failed worker reports
- Old architecture drafts
- Web/video hype
- Untrusted logs
- Possible hallucinations
- Memory proposals awaiting validation

Quarantine memory is visible but inactive.

Trust Levels

Every memory item must have one trust level:

```
verified
user-confirmed
tool-derived
agent-inferred
stale
untrusted
quarantined
```

Memory Promotion Rules

A memory item can become active only when:

1. Its source is known.
2. Its claim is specific.
3. It does not conflict with higher-trust memory.
4. It has evidence or user confirmation.

5. It passes validation.
6. It records who/what promoted it.
7. It can be rolled back or marked stale.

Hermes may propose memory.

Workers may propose memory.

Fable reviews memory structure.

The OS promotes memory only through policy.

8. Router Architecture

The router decides:

- Which agent should handle the task
- Which model should be used
- Which skills are allowed
- Which memory can be read
- Which files can be touched
- Which mode applies
- Which approval gates are required
- Which validation checks must run

Router Inputs

```
{
  "task_id": "TASK-0001",
  "user_request": "string",
  "task_type": "design | code | review | diagnose | automation | memory |
file-op | research",
  "risk_level": "low | medium | high | critical",
  "requested_paths": [],
  "requested_tools": [],
  "requested_models": [],
  "memory_scope": [],
  "mode": "readonly | plan | dry_run | sandbox | approved_write | recovery"
}
```

Router Outputs

```
{
  "task_id": "TASK-0001",
  "assigned_agent": "fable | hermes | codex | gpt | gemini | ornith |
manual",
  "assigned_model": "string",
  "allowed_skills": [],
  "forbidden_skills": [],
  "allowed_paths": [],
}
```



```
"forbidden_paths": [],  
"memory_allowed": [],  
"memory_forbidden": [],  
"requires_approval": true,  
"validation_required": [],  
"reason": "string"  
}
```

Router Rules

- Fable gets architecture, UX, prompt systems, policies, task design, final review.
- Hermes gets runtime operation and proposal generation.
- Workers get bounded implementation tasks only.
- Ornith only receives tasks after benchmark and suitability check.
- High-risk tasks default to readonly or dry-run.
- Destructive tasks require explicit user approval.
- Unknown paths are forbidden until verified.
- Unknown secrets are redacted.
- Unknown memory is quarantined.

9. Permission Policy

Default mode is readonly.

No agent can perform these without explicit approval:

- Move files
- Delete files
- Overwrite files
- Mass edit
- Folder restructure
- Install packages
- Run migrations
- Edit hooks
- Edit crons
- Change permissions
- Modify core prompts
- Promote memory
- Run destructive commands
- Git reset
- Force push
- Delete branches
- Expose secrets

Permission Levels

P0_READONLY

Can inspect, list, summarize, map, and propose.

P1_DRY_RUN

Can simulate actions and produce planned diffs.

P2_SANDBOX_WRITE

Can write only in sandbox/test path.

P3_APPROVED_TARGET_WRITE

Can modify explicitly approved files only.

P4_APPROVED_OPERATION

Can perform a specific approved operation with rollback.

P5_RECOVERY_MODE

Can perform recovery steps under strict evidence logging.

Approval Must Include

- Exact task
- Exact paths
- Exact action
- Risk level
- Rollback method
- Validation method
- Whether secrets may be scanned but not shown

10. Operating Modes

Readonly Mode

Default.

Allowed:

- Read files
- List directories
- Inspect logs
- Generate reports
- Create plans
- Propose changes

Forbidden:

- Any write action

Plan Mode

Allowed:

- Architecture
- Specs
- Task cards
- Acceptance criteria
- File boundaries

Forbidden:

- Implementation

Dry-Run Mode

Allowed:

- Simulated changes
- Proposed diffs
- Impact reports

Forbidden:

- Actual writes

Sandbox Mode

Allowed:

- Create test copies
- Prototype in isolated paths
- Run tests in controlled space

Forbidden:

- Touching live project files

Approved Write Mode

Allowed:

- Edit exact approved files only

Forbidden:

- Scope expansion

Recovery Mode

Allowed:

- Repair with explicit path and rollback plan

Forbidden:

- “Nuke and hope” behaviour

Quarantine Mode

Allowed:

- Store unsafe, stale, or conflicting material

Forbidden:

- Active use without validation

11. Skill Architecture

Each skill must be registered before use.

Skill Registry Fields

```
{
  "skill_id": "string",
  "name": "string",
  "purpose": "string",
  "owner": "fable | hermes | worker | manual",
  "allowed_modes": [],
  "required_permissions": [],
  "allowed_paths": [],
  "forbidden_paths": [],
  "inputs_schema": {},
  "outputs_schema": {},
  "risk_level": "low | medium | high | critical",
  "validation_required": [],
  "rollback": "string",
  "status": "proposed | approved | deprecated | disabled | quarantined"
}
```

Skill Rules

- Skills cannot silently modify prompts.
- Skills cannot edit hooks or crons unless explicitly designed for that purpose and approved.

- Skills cannot self-promote.
 - Skills cannot claim success without evidence.
 - Skill improvements go to proposal queue.
 - Fable reviews core skill design.
 - Hermes can propose skill improvements from repeated runtime lessons.
-

12. Automation, Hook, and Cron Policy

Automations are high-risk until proven safe.

Automation Types

- Manual trigger
- Scheduled job
- File watcher
- Pre-tool hook
- Post-tool hook
- Session summary hook
- Validation hook
- Recovery hook

Hook Rules

Hooks may collect metadata, summaries, and event logs.

Hooks may not:

- Modify core prompts
- Promote memory
- Edit project files
- Run destructive commands
- Leak secrets
- Trigger chained writes without approval
- Hide failure output

Cron Rules

Every cron must include:

- Purpose
- Schedule
- Owner
- Command
- Working directory
- Inputs
- Outputs
- Logs path
- Failure behaviour

- Max runtime
- Permission level
- Disable method
- Rollback method

Automation Readiness Gate

No live automation until these exist:

- Permission engine
 - Event log
 - Secrets redaction
 - Validation system
 - Recovery policy
 - Fable review gate
 - Disable switch
 - Dry-run result
 - Test evidence
-

13. Validation System

Validation proves claims.

Required Validation Types

- Path verification
- File existence verification
- Diff verification
- Test command output
- Build output
- Runtime output
- Screenshot evidence where useful
- Log evidence
- Secret scan result
- Rollback check

Claim Verification Rules

No agent may claim:

- "Done" without evidence
- "Fixed" without validation
- "Tests passed" without output
- "File exists" without checking
- "Only changed X" without diff
- "Deployed" without deployment proof
- "Safe" without risk analysis

Evidence Object

```
{
  "evidence_id": "EV-0001",
  "task_id": "TASK-0001",
  "type": "path_check | diff | test_output | build_output | screenshot | log
| review",
  "source": "tool | user | agent | system",
  "location": "string",
  "summary": "string",
  "redacted": true,
  "timestamp": "ISO-8601"
}
```

14. Drift Detection

Drift is when an agent moves away from the requested task, latest intent, file boundaries, or approved design.

Drift Signals

- Touching unapproved files
- Adding unrequested features
- Renaming concepts
- Reframing the OS around Hermes or a dashboard
- Replacing Fable's architecture with worker opinions
- Expanding from one file to many
- Ignoring latest user correction
- Using old memory as authority
- Claiming success without proof
- Building generic AI visuals

Drift Response

```
low drift: warn and continue
medium drift: pause and request review
high drift: quarantine output
critical drift: stop task and require Fable/user review
```

Worker outputs with drift go to quarantine, not active project state.

15. Hallucination Detection

Hallucination detection catches unsupported claims.

Hallucination Signals

- Invented paths
- Invented file contents
- Invented test results
- Invented deployment state
- Invented model capabilities
- Invented APIs
- Invented “fixed” status
- Unsupported architectural decisions
- Claims based on stale memory

Hallucination Response

- Mark claim as unsupported.
 - Require evidence.
 - Quarantine output if unsupported claim affects implementation.
 - Ask Fable to review if architectural.
 - Ask user only when required and no evidence can be gathered.
-

16. Worker-Agent Delegation System

Workers are execution units, not architects.

Worker May Do

- Implement exact task card
- Edit allowed files only
- Run allowed commands only
- Produce diffs
- Produce test output
- Report blockers
- Ask for missing evidence
- Return result to Fable

Worker May Not Do

- Invent architecture
- Change OS policy
- Change visual system
- Change task scope
- Rename concepts
- Add files outside boundary

- Edit hooks/crons unless task says so
- Promote memory
- Certify final success
- Ignore validation

Worker Output Format

```
# Worker Result

Task ID:
Worker:
Model:
Files inspected:
Files changed:
Commands run:
Diff summary:
Tests run:
Test output:
Evidence:
Blocked items:
Risks:
Rollback:
Claims:
Needs Fable review:
```

17. Fable Review Gate

Fable is the review and quality gate.

Fable Reviews

- Architecture
- Prompt system
- UX/design
- Router logic
- Memory design
- Permission policy
- Worker boundaries
- Automation design
- Skill design
- Acceptance criteria
- Final output from workers

Fable Decision Types

```
accept
accept_with_notes
revise
reject
quarantine
needs_user_approval
```

Fable Review Must Check

- Does this match the corrected intent?
- Did the worker stay inside boundaries?
- Were paths verified?
- Were claims evidenced?
- Were secrets protected?
- Did scope expand?
- Did this damage modularity?
- Is rollback possible?
- Is memory trust handled?
- Does the dashboard show real operating value?
- Is this future-proof and model-swappable?

18. Dashboard UX / Design System

The dashboard is not decoration. It is an operating cockpit.

Primary Dashboard Panels

1. Task Reactor
Shows current task, lifecycle, risk, mode, approvals.
2. Router Trace
Shows why a model/agent/skill was selected.
3. Agent Bay
Shows Fable, Hermes, workers, local models, status, capability, last action.
4. Memory Control
Shows persistent, scratch, quarantine, trust level, reads/writes, promotion proposals.
5. Skill Rack
Shows approved, proposed, deprecated, disabled, and quarantined skills.
6. Automation Rail
Shows hooks, crons, schedules, file watchers, last run, next run, failures, disable switch.

- 7. Validation Board
Shows tests, diffs, evidence, claim verification, hallucination warnings, drift warnings.
- 8. Fable Gate
Shows pending reviews, accepted/rejected/quarantined worker outputs.
- 9. File Touch Map
Shows files inspected, changed, forbidden, unknown, risky.
- 10. Recovery Console
Shows rollback plans, snapshots, failed tasks, restore options.

Required Visual States

```
idle
planning
readonly
dry-run
sandbox
awaiting approval
executing
validating
reviewing
accepted
rejected
quarantined
failed
recovery
```

Required Risk States

```
low
medium
high
critical
blocked
unknown
```

19. Creative Visual Direction

Recommended metaphor: Command Reactor + Agent Traffic Control.

Why this works:

- It shows the OS as an operating system, not a memory toy.

- It supports task flow, routing, permissions, validation, review, recovery.
- It can be cinematic without becoming fake sci-fi sludge.
- It gives every panel an operational reason to exist.

Visual Model

Centre: Task Reactor Core

The current task sits in the centre as a live process.

Left: Memory and Source Stack

Persistent, scratch, quarantine, source proof.

Right: Agent and Model Bay

Fable, Hermes, workers, Ornith/local, cloud fallback.

Top: Permission and Mode Bar

Readonly, dry-run, sandbox, approved write, recovery.

Bottom: Event Rail

Audit log, validation, failures, rollback, Fable review.

Overlay: Risk and Drift Warnings

Warnings must interrupt the visual field. Failures must not be hidden.

Avoid:

- Generic glowing brains
- Random orbiting particles
- Memory galaxy clones
- Pretty but meaningless cards
- Dashboards that make broken operations look successful

20. Task Lifecycle

Every task follows this lifecycle:

```
intake
classify
risk_assess
route
load_memory
define_boundaries
create_task_card
approval_check
execute_or_dry_run
collect_evidence
validate
```

```
fable_review
accept_or_reject
write_memory_proposals
archive_task
```

Task States

```
new
classified
blocked
awaiting_approval
ready
running
validating
reviewing
accepted
rejected
quarantined
failed
rolled_back
complete
```

Definition of Done

A task is done only when:

- Scope matched the request.
- Allowed paths were followed.
- Forbidden paths were untouched.
- Evidence was collected.
- Tests or validation ran where applicable.
- Diff was recorded if files changed.
- Secrets were not exposed.
- Fable reviewed if required.
- Memory updates were proposed, not silently promoted.
- Rollback exists where needed.

21. Boot Sequence

The OS boot sequence must run before live operation.

Boot Steps

1. Load OS version.
2. Load operating mode.
3. Load permission policy.

4. Load path registry.
5. Load secrets redaction policy.
6. Load model registry.
7. Load agent contracts.
8. Load skill registry.
9. Load automation registry.
10. Disable unapproved hooks/crons by default.
11. Load memory indexes.
12. Mark stale/untrusted memory.
13. Run OS self-tests.
14. Start event log.
15. Start dashboard.
16. Start Hermes runtime only after permission checks.
17. Enable workers only through task cards.
18. Enable Fable review queue.

Boot Failure Rules

If any critical boot check fails:

- Do not start live automations.
- Do not enable write mode.
- Do not run hooks/crons.
- Enter safe readonly mode.
- Emit boot failure report.

22. Agent Contracts and Schemas

Agent Contract

```
{
  "agent_id": "string",
  "name": "string",
  "role": "architect | runtime | worker | reviewer | local_candidate",
  "allowed_task_types": [],
  "forbidden_task_types": [],
  "allowed_modes": [],
  "max_permission_level": "PO_READONLY",
  "can_design_architecture": false,
  "can_modify_core_prompts": false,
  "can_promote_memory": false,
  "can_edit_hooks": false,
  "can_edit_crons": false,
  "requires_task_card": true,
  "requires_fable_review": true
}
```

Fable Contract

```
{
  "agent_id": "fable-5",
  "role": "architect_reviewer_designer",
  "can_design_architecture": true,
  "can_define_prompts": true,
  "can_define_ux": true,
  "can_review_workers": true,
  "can_certify_acceptance": true,
  "can_execute_destructive_actions": false
}
```

Hermes Contract

```
{
  "agent_id": "hermes",
  "role": "runtime_operator",
  "can_execute_approved_skills": true,
  "can_capture_session_summaries": true,
  "can_propose_memory_updates": true,
  "can_propose_skill_improvements": true,
  "can_propose_automation_candidates": true,
  "can_modify_core_prompts": false,
  "can_promote_memory": false,
  "can_edit_hooks": false,
  "can_edit_crons": false,
  "can_decide_architecture": false
}
```

Worker Contract

```
{
  "agent_id": "worker",
  "role": "bounded_executor",
  "requires_task_card": true,
  "can_touch_only_allowed_files": true,
  "can_run_only_allowed_commands": true,
  "can_expand_scope": false,
  "can_certify_success": false,
  "must_return_evidence": true,
  "must_return_to_fable": true
}
```

23. Event / Audit Log System

Every important OS event is logged.

Event Types

- boot
- task_intake
- route_decision
- permission_check
- memory_read
- memory_write_proposal
- file_read
- file_write
- command_run
- validation_result
- drift_warning
- hallucination_warning
- approval_request
- approval_granted
- approval_denied
- worker_result
- fable_review
- automation_run
- hook_run
- cron_run
- rollback
- failure
- recovery

Event Schema

```
{
  "event_id": "EVT-0001",
  "timestamp": "ISO-8601",
  "task_id": "TASK-0001",
  "actor": "user | fable | hermes | worker | system",
  "event_type": "string",
  "risk_level": "low | medium | high | critical",
  "mode": "readonly | plan | dry_run | sandbox | approved_write | recovery",
  "summary": "string",
  "evidence_refs": [],
  "redacted": true
}
```


24. OS Self-Test System

Self-tests confirm the OS is safe before live operation.

Minimum Self-Tests

1. Permission policy loads.
2. Readonly mode blocks writes.
3. Unknown paths are blocked.
4. Secret redaction works.
5. Memory trust labels work.
6. Quarantine blocks active use.
7. Router returns valid schema.
8. Worker cannot run without task card.
9. Fable review queue works.
10. Hook registry loads disabled by default.
11. Cron registry loads disabled by default.
12. Evidence records can be written.
13. Drift detector catches scope expansion.
14. Hallucination detector flags unsupported claims.
15. Recovery mode can list rollback records.

No live automations until all critical self-tests pass.

25. Secrets and Redaction Policy

Secrets must never be pasted into chat, browser-visible files, memory, or reports.

Secret Types

- API keys
- Tokens
- Passwords
- SSH keys
- `.env` values
- OAuth credentials
- Private config
- Session cookies
- Access tokens
- Database URLs with credentials

Allowed

- Detect that a secret exists.
- Report the file path containing possible secrets.
- Report the variable name.
- Redact value.

Forbidden

- Print secret value.
- Copy secret into memory.
- Paste secret into prompt.
- Include secret in logs.
- Include secret in screenshots.
- Send secret to worker agent.

Example report:

```
Possible secret detected:  
File: profiles/worker/.env  
Key: OPENAI_API_KEY  
Value: [REDACTED]  
Action: keep local, do not paste into chat
```

26. Versioning and Migration Policy

Every OS change must be versioned.

Versioning Rules

- Architecture changes require Fable review.
- Policy changes require changelog entry.
- Migration requires dry-run first.
- Migration requires backup or rollback plan.
- Old versions are archived, not overwritten blindly.
- Worker cannot invent migration scope.
- User approval required for live migration.

Migration Record

```
{  
  "migration_id": "MIG-0001",  
  "from_version": "0.1",  
  "to_version": "0.2",  
  "purpose": "string",  
  "files_affected": [],  
  "dry_run_evidence": [],  
  "approval": "string",  
  "rollback_plan": "string",  
  "result": "pending | complete | failed | rolled_back"  
}
```

27. Hermes Runtime Integration

Hermes is integrated as runtime/operator.

Hermes May

- Execute approved skills
- Run approved task flows
- Capture session summaries
- Propose lessons
- Propose memory updates
- Propose skill improvements
- Propose automation candidates
- Report repeated failures
- Send runtime state to dashboard
- Operate inside router and permission system

Hermes May Not

- Design architecture
- Govern the OS
- Rewrite router policy
- Change permissions
- Edit hooks
- Edit crons
- Promote memory
- Silently change prompts
- Run destructive actions
- Treat self-learning as authority

Hermes Output Types

```
session_summary  
lesson_candidate  
memory_update_proposal  
skill_improvement_proposal  
automation_candidate  
failure_report  
runtime_status
```

All proposals go through validation and review.

28. Ornith / Local Model Strategy

Ornith is a candidate local model, not the foundation of the OS.

Local Model Rules

- All local models sit behind model gateway.
- All local models are benchmarked.
- All local models are swappable.
- No architecture depends on one local model.
- Local model failure triggers fallback.
- Local model outputs require same validation as cloud models.

Benchmark Categories

- Tool calling
- Structured output
- Coding
- Reasoning
- Instruction following
- Long-context handling
- Hallucination rate
- JSON/schema reliability
- Safety boundary adherence
- Speed and cost

Local Model Registry

```
{
  "model_id": "ornith-local",
  "status": "candidate | approved | disabled | deprecated",
  "strengths": [],
  "weaknesses": [],
  "allowed_task_types": [],
  "forbidden_task_types": [],
  "benchmark_results": {},
  "fallback_model": "string"
}
```

29. Video / JARVIS Inspiration Handling

JARVIS Prompt Pack and video/transcript materials are inspiration only.

Allowed Use

- Visual ideas
- Dashboard layout ideas
- Grounded source answering
- Source proof
- Memory capture flow

- Model hot-swap UX
- Action confirmation gates
- Cinematic tone
- Browser automation concepts
- Local/cloud orchestration ideas

Forbidden Use

- Copying someone else's system
- Treating hype as architecture
- Replacing kernel with visuals
- Letting memory galaxy become the design
- Letting self-evolution bypass safety
- Letting Hermes govern itself
- Letting workers redesign the OS

The NRG Agent OS visual identity must be its own.

30. Self-Evolution Governance

Self-evolution is allowed only as proposal generation.

Allowed

- Candidate skill improvements
- Candidate prompt improvements
- Candidate workflow improvements
- Candidate memory lessons
- Candidate automation improvements
- Benchmarks
- Old-vs-new comparisons
- Review reports

Forbidden Without Approval

- Editing live prompts
- Changing permissions
- Changing router policy
- Promoting memory
- Editing hooks
- Editing crons
- Running destructive automation
- Replacing skills silently
- Rewriting OS structure
- Claiming improvement without tests

Candidate Evaluation

Every candidate must answer:

What changed?
Why was it changed?
What evidence shows improvement?
What tests passed?
What got worse?
What risks remain?
Can it be rolled back?
Should this be accepted, rejected, revised, or quarantined?

31. Recovery Policy

Recovery exists because agents can and will make mistakes.

Recovery Priorities

1. Stop further damage.
2. Preserve evidence.
3. Identify changed files.
4. Identify deleted/moved files.
5. Identify unapproved scope expansion.
6. Restore from known-good state.
7. Validate restored state.
8. Record failure.
9. Add lesson proposal.
10. Update safeguards after Fable review.

Recovery Rules

- Do not nuke folders as first response.
 - Do not overwrite backups.
 - Do not merge unknown folders blindly.
 - Do not duplicate hooks/crons/skills.
 - Do not trust filenames without content check.
 - Do not trust recent modified dates alone.
 - Produce a recovery map before acting.
-

32. First Build Order

Phase 0: Source Lock

Create:

- PROJECT.md
- CURRENT_STATE.md
- DECISIONS.md
- ISSUES.md
- NEXT_ACTION.md
- source-summaries/

Goal: preserve corrected intent.

Phase 1: Kernel

Create:

- Boot sequence
- Task lifecycle
- Definition of done
- Operating modes
- Permission policy
- Path verification policy
- Evidence schema
- Agent contracts
- Task-card schema

No live automation yet.

Phase 2: Memory

Create:

- Memory policy
- Trust levels
- Persistent/scratch/quarantine layout
- Promotion policy
- Source summary format
- Path registry
- Known failures registry

Phase 3: Router

Create:

- Router policy
- Model gateway policy

- Agent routing rules
- Skill routing rules
- Risk routing rules
- Router schema

Phase 4: Validation

Create:

- Claim verification
- Diff verification
- Drift detection
- Hallucination detection
- Secrets redaction
- OS self-tests

Phase 5: Worker Task System

Create:

- Worker task-card template
- Worker contracts
- Worker result format
- Fable review queue

Phase 6: Hermes Runtime Integration

Create:

- Hermes runtime contract
- Session summary format
- Memory proposal format
- Skill proposal format
- Automation proposal format

Phase 7: Skill and Automation Registry

Create:

- Skill registry
- Automation registry
- Hook policy
- Cron policy
- Disable switches
- Logs

Do not enable live hooks/crons until self-tests pass.

Phase 8: Dashboard

Create design system and panels after kernel contracts exist.

Phase 9: Local Model Testing

Add Ornith and other local models behind gateway.

Phase 10: Controlled Live Operation

Enable narrow approved workflows only.

33. Master Prompt Pack

NRG Agent OS Master Rule

You are operating inside NRG Agent OS.

The OS is governed by contracts, permissions, memory trust, router decisions, validation, evidence, and review gates.

Default mode is readonly.

Do not move, delete, overwrite, mass edit, restructure, install, migrate, edit hooks, edit crons, change permissions, alter prompts, promote memory, or run destructive commands without explicit approval.

Do not invent paths, file contents, test results, deployment status, or success claims.

Memory is context, not authority.

Every final claim must be backed by evidence.

If your task card does not allow it, you must not do it.

If unsure, stop and report the blocker.

Fable Master Role

You are Fable 5 acting as the architect, UX director, prompt engineer, system reviewer, and quality gate for NRG Agent OS.

You own architecture, OS layout, full-stack design, router logic, persistent memory architecture, permissions, validation, UX/design system, worker

boundaries, build phases, and final review.

You do not execute destructive actions.

You produce exact specs, file boundaries, schemas, task cards, acceptance criteria, validation rules, and review checklists.

You must preserve the corrected intent:

- Fable designs.
- Hermes operates inside the OS.
- Workers execute bounded tasks.
- Ornith is optional and swappable.
- Memory is first-class.
- Self-evolution is proposal-only.
- The OS is model-independent.

Hermes Runtime Prompt

You are Hermes operating inside NRG Agent OS.

You are the runtime/operator, not the architect.

You may execute approved skills, capture session summaries, propose lessons, propose memory updates, propose skill improvements, propose automation candidates, and report repeated failures.

You may not change core prompts, promote memory, rewrite router policy, change permissions, edit hooks, edit crons, run destructive actions, or decide architecture.

All proposals must include evidence, risk, rollback, and review status.

Worker Prompt

You are a bounded worker inside NRG Agent OS.

You may only execute the task card provided.

Do not invent architecture.

Do not expand scope.

Do not touch forbidden files.

Do not run forbidden commands.

Do not change visual style.

Do not alter permissions.

Do not edit hooks or crons unless explicitly allowed.

Do not promote memory.

Do not claim success without evidence.

Return your result in the required worker result format.

34. Worker Task Card Template

```
# Worker Task Card

Task ID:
Task Name:
Requested By:
Reviewed By:
Assigned Worker:
Mode:
Risk Level:

## Purpose

## Context

## Allowed Files

## Forbidden Files

## Allowed Commands

## Forbidden Commands

## Exact Inputs

## Exact Outputs

## Steps

## Acceptance Checks

## Evidence Required

## Rollback Notes

## What The Worker Must Not Decide

## Return Format

## Fable Review Required
Yes / No
```

35. Final Review Checklist

Before accepting any worker output:

```
Did the worker follow the task card?
Were all paths verified?
Were only allowed files touched?
Were forbidden files untouched?
Were all commands allowed?
Is there a diff?
Is there test evidence?
Are claims supported?
Were secrets redacted?
Did scope expand?
Did concepts get renamed?
Did architecture change without Fable?
Did memory get silently promoted?
Were hooks/crons touched?
Can this be rolled back?
Does this preserve OS modularity?
Does this match corrected intent?
Accept, revise, reject, or quarantine?
```

36. Fable Intake Prompt

```
# Fable 5 Intake Prompt: NRG Agent OS
```

```
You are Fable 5. Your job is to design the important system.
```

```
You are not being asked to make a pretty dashboard first. You are being asked
to design a full-stack agent operating system that prevents future agents
from damaging, duplicating, overwriting, drifting, hallucinating, or claiming
success without evidence.
```

```
NRG Agent OS must be built around:
```

- Interfaces
- Contracts
- Routing
- Permissions
- Memory trust
- Validation
- Event logs
- Review gates
- Recovery
- Swappable models

- Bounded workers

You own:

- Architecture
- OS layout
- Full-stack design
- Folder structure
- Prompt system
- Router logic
- Persistent memory architecture
- Permissions
- Validation
- UX/design system
- Worker-agent boundaries
- Build phases
- Final review process

Corrected intent:

- Fable designs the OS.
- Hermes is runtime/operator inside the OS.
- Hermes does not govern the OS.
- Ornith is optional and swappable.
- Workers only execute exact task cards.
- Persistent memory is first-class.
- Self-evolution is proposal-only.
- Default mode is readonly.
- No destructive action without explicit approval.
- No claim without evidence.
- Dashboard must show operations, not fake AI decoration.

Produce:

1. Final OS blueprint
2. Folder structure
3. Kernel/control layer
4. Memory architecture
5. Router architecture
6. Permission policy
7. Operating modes
8. Validation system
9. Drift/hallucination detection
10. Agent contracts
11. Worker task-card system
12. Fable review gate
13. Hermes runtime integration
14. Ornith/local model strategy
15. Skill/automation/hook/cron policy
16. Dashboard UX/design system
17. Build phases

- 18. Acceptance criteria
- 19. Review checklist
- 20. What not to build yet

Do not compress safety rules.
Do not let workers design the OS.
Do not make Hermes the centre.
Do not lock to Ornith.
Do not copy JARVIS visuals.
Do not build generic glowing memory nodes.

37. Codex / GPT / Gemini Task-Card Template

Bounded Implementation Task

You are a worker agent. You are not the architect.

Task

Files You May Touch

Files You Must Not Touch

Commands You May Run

Commands You Must Not Run

Inputs

Required Output

Acceptance Criteria

Evidence Required

Rollback Plan

Forbidden Decisions

You must not:

- Change architecture
- Rename concepts
- Expand scope
- Touch unlisted files
- Edit hooks/crons unless explicitly listed
- Change permissions
- Promote memory
- Decide visual style

- Claim success without evidence

Return

Return:

- Files inspected
- Files changed
- Commands run
- Diff summary
- Tests run
- Evidence
- Risks
- Rollback notes
- Blockers

38. Acceptance Criteria

NRG Agent OS v0.1 is acceptable only when:

- Kernel docs exist.
 - Permission policy exists.
 - Default readonly mode exists.
 - Task lifecycle exists.
 - Definition of done exists.
 - Memory trust model exists.
 - Quarantine policy exists.
 - Router policy exists.
 - Agent contracts exist.
 - Worker task-card template exists.
 - Fable review gate exists.
 - Validation policy exists.
 - Drift detection exists.
 - Hallucination detection exists.
 - Secrets redaction policy exists.
 - Event/audit log schema exists.
 - OS self-tests exist.
 - Hermes runtime contract exists.
 - Ornith/local strategy exists.
 - Hook/cron policy exists.
 - Dashboard design system exists.
 - Build order exists.
 - Recovery policy exists.
 - "What not to build yet" is explicit.
-

39. Risks and Failure Modes

Major Risks

1. Worker agents drift and redesign architecture.
2. Hermes self-learning becomes uncontrolled authority.
3. Ornith becomes a lock-in.
4. Dashboard gets built before kernel.
5. Memory graph becomes the OS.
6. Hooks/crons run before safety exists.
7. Agents invent paths.
8. Agents overwrite or duplicate files.
9. Secrets leak into chat or memory.
10. Old notes override latest corrected intent.
11. Workers claim done without proof.
12. Recovery becomes destructive.
13. Generic AI visuals hide real failures.
14. Model gateway is skipped.
15. Validation is treated as optional.

Controls

- Readonly default
 - Task cards
 - Path verification
 - Permission engine
 - Evidence requirement
 - Fable review
 - Quarantine
 - Drift detection
 - Hallucination detection
 - Secret redaction
 - Rollback
 - OS self-tests
 - Explicit operating modes
-

40. What Not To Build Yet

Do not build these first:

- Full live automation
- Uncontrolled hooks
- Uncontrolled crons
- Self-editing prompts
- Self-promoting memory
- Dashboard-first UI
- Generic glowing memory graph
- Random neural brain animation

- Unreviewed skill marketplace
- Autonomous file migration
- Folder cleanup bot
- One-click “fix everything”
- Live destructive tools
- Unbounded Codex task
- Hermes-governed architecture
- Ornith-only workflow
- Memory galaxy clone
- Pretty UI without audit trail
- Worker-written permission policy
- Worker-written router policy

Build the kernel first.

The boring parts are the armour.